# VĂN PHẠM PHI NGỮ CẢNH CỦA NGÔN NGỮ UPL

24021485 Lã Việt Hoàng
24021533 Nguyễn Đức Khiêm

## 1. Giới thiệu

Tài liệu này đặc tả văn phạm phi ngữ cảnh (CFG) của ngôn ngữ lập trình UPL. Văn phạm là bộ bốn $G = (N, T, P, S)$, trong đó $N$ là tập ký hiệu không kết thúc, $T$ là tập ký hiệu kết thúc (token), $P$ là tập luật sinh và $S = \textit{Program}$ là ký hiệu bắt đầu.

## 2. Các giả định thiết kế

Đề bài chưa quy định một số chi tiết, nên văn phạm này thống nhất như sau:

1. Câu lệnh `for` có dạng kiểu C: `for (khởi_tạo; điều_kiện; cập_nhật) { … }`. Phần khởi tạo là khai báo hoặc phép gán, phần cập nhật là phép gán.
2. Bổ sung hai hằng boolean `true` và `false`.
3. Thân của `if`, `else`, `do-while`, `for` bắt buộc nằm trong cặp `{ }`.
4. Phép so sánh (`>`, `>=`, `==`) không có tính kết hợp, nghĩa là `a > b > c` không hợp lệ.
5. Văn phạm chỉ mô tả cú pháp. Các ràng buộc ngữ nghĩa (khai báo trước khi dùng, khai báo lại, tương thích kiểu) không thuộc phạm vi tài liệu này.

## 3. Ký hiệu kết thúc (token)

| Nhóm | Token | Mô tả |
|---|---|---|
| Từ khóa | `begin end int bool if then else do while for print true false` | Từ khóa dành riêng, không dùng làm định danh |
| Định danh | `id` | Bắt đầu bằng chữ cái, nếu có chữ số thì chỉ nằm ở cuối: `[A-Za-z]+[0-9]*` |
| Hằng số nguyên | `num` | Dãy một hoặc nhiều chữ số: `[0-9]+` |
| Toán tử số học | `+ *` | Cộng, nhân |
| Toán tử so sánh | `> >= ==` | Lớn hơn, lớn hơn hoặc bằng, bằng |
| Phép gán | `=` | Gán giá trị |
| Phân cách | `( ) { } ;` | |

Khoảng trắng và comment kiểu Java (`//` … đến cuối dòng, `/*` … `*/` có thể nhiều dòng) được bộ phân tích từ vựng loại bỏ, không xuất hiện trong văn phạm.

## 4. Ký hiệu không kết thúc

| *Ký hiệu* | **Ý nghĩa** |
|---|---|
| *Program* | Toàn bộ chương trình (ký hiệu bắt đầu) |
| *StmtList* | Danh sách câu lệnh (có thể rỗng) |
| *Stmt* | Một câu lệnh |
| *Block* | Khối lệnh trong `{ }` |
| *Decl* | Khai báo biến (có thể kèm khởi tạo) |
| *Type* | Kiểu dữ liệu |
| *Assign* | Câu lệnh gán |
| *Print* | Câu lệnh in |
| *IfStmt* | Câu lệnh rẽ nhánh |
| *DoWhile* | Câu lệnh lặp do-while |
| *ForStmt* | Câu lệnh lặp for |
| *ForInit* | Phần khởi tạo của for |
| *Expr* | Biểu thức (có thể có so sánh) |
| *RelOp* | Toán tử so sánh |
| *Arith* | Biểu thức cộng |
| *Term* | Biểu thức nhân |
| *Factor* | Toán hạng cơ bản |

## 5. Các luật sinh

(1) *Program* → `begin` *StmtList* `end`

(2) *StmtList* → *StmtList Stmt*
(3) | $\varepsilon$

(4) *Stmt* → *Decl* `;`
(5) | *Assign* `;`
(6) | *Print* `;`
(7) | *IfStmt*
(8) | *DoWhile*
(9) | *ForStmt*
(10) | *Block*

(11) *Block* → `{` *StmtList* `}`

(12) *Decl* → *Type* `id`
(13) | *Type* `id` = *Expr*
(14) *Type* → `int`
(15) | `bool`
(16) *Assign* → `id` = *Expr*
(17) *Print* → `print` `(` *Expr* `)`

(18) *IfStmt* → `if` `(` *Expr* `)` `then` *Block*
(19) | `if` `(` *Expr* `)` `then` *Block* `else` *Block*
(20) *DoWhile* → `do` *Block* `while` `(` *Expr* `)` `;`
(21) *ForStmt* → `for` `(` *ForInit* `;` *Expr* `;` *Assign* `)` *Block*
(22) *ForInit* → *Decl*
(23) | *Assign*

(24) *Expr* → *Arith*
(25) | *Arith RelOp Arith*
(26) *RelOp* → `>`
(27) | `>=`
(28) | `==`

(29) *Arith* → *Arith* + *Term*
(30) | *Term*
(31) *Term* → *Term* * *Factor*
(32) | *Factor*

(33) *Factor* → `id`
(34) | `num`
(35) | `true`
(36) | `false`
(37) | `(` *Expr* `)`

## 6. Độ ưu tiên và tính kết hợp

| Mức (cao đến thấp) | Toán tử | Kết hợp | Được thể hiện bởi |
|---|---|---|---|
| 1 | `( )` | | *Factor* → `(` *Expr* `)` |
| 2 | `*` | Trái | *Term* → *Term* `*` *Factor* |
| 3 | `+` | Trái | *Arith* → *Arith* `+` *Term* |
| 4 | `> >= ==` | Không kết hợp | *Expr* → *Arith RelOp Arith* |

- Toán tử nằm càng sâu trong chuỗi *Expr* → *Arith* → *Term* → *Factor* thì ưu tiên càng cao. Vì `*` nằm sâu hơn `+` nên `*` ưu tiên hơn `+`.
- Đệ quy trái làm `+` và `*` kết hợp trái: `a+b+c` được hiểu là `(a+b)+c`.
- Luật (25) chỉ chứa một *RelOp* nên không thể sinh `a>b>c`.
- Dấu ngoặc ở *Factor* cho phép đặt lại thứ tự tính toán.

## 7. Tính không nhập nhằng

- **Dangling else:** thân của `if` luôn là *Block* (bắt đầu bằng `{` và kết thúc bằng `}`), nên `else` luôn thuộc về `if` gần nhất chưa có `else` mà không cần quy tắc phụ.
- **Biểu thức:** phân tầng *Expr*/*Arith*/*Term*/*Factor* cho mỗi biểu thức đúng một cây phân tích.
- **Câu lệnh:** mỗi dạng câu lệnh bắt đầu bằng một token khác nhau (`int`/`bool`, `id`, `print`, `if`, `do`, `for`, `{`) nên không có hai luật cùng dẫn đến một chuỗi.

## 8. Ví dụ dẫn xuất

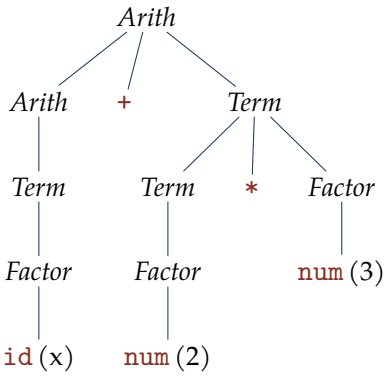
**Ví dụ 1.** Chương trình rỗng `begin end`:

$$\textit{Program} \Rightarrow \texttt{begin}\ \textit{StmtList}\ \texttt{end} \Rightarrow \texttt{begin}\ \varepsilon\ \texttt{end}$$

**Ví dụ 2.** Câu lệnh `int y = x + 1 ;` (dẫn xuất trái nhất):

$$\begin{aligned}
\textit{Stmt} &\Rightarrow \textit{Decl}\ \texttt{;} \\
&\Rightarrow \textit{Type}\ \texttt{id} = \textit{Expr}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \textit{Expr}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \textit{Arith}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \textit{Arith} + \textit{Term}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \textit{Term} + \textit{Term}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \textit{Factor} + \textit{Term}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \texttt{id} + \textit{Term}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \texttt{id} + \textit{Factor}\ \texttt{;} \\
&\Rightarrow \texttt{int id} = \texttt{id} + \texttt{num}\ \texttt{;}
\end{aligned}$$

**Ví dụ 3.** Thứ tự ưu tiên của `x + 2 * 3`. Cây phân tích thể hiện `2 * 3` được kết hợp trước:




**Ví dụ 4.** Một chương trình có rẽ nhánh và vòng lặp:

```
begin
  int x = 1;
  if (x >= 1) then { print(x); } else { x = x + 1; }
  do { x = x * 2; } while (x > 10);
end
```

Chương trình trên được sinh như sau:

- Luật (1) cho khung `begin` ... `end`.
- Các luật (2), (4), (13) cho khai báo `int x = 1;`.
- Các luật (7), (19), (25), (27) cho `if` ... `then` ... `else` ... với phép so sánh `>=`.
- Các luật (8), (20), (25), (26) cho vòng `do` ... `while`.

## 9. Đối chiếu với chương trình mẫu của đề

| **Đoạn mã trong đề** | **Luật áp dụng** |
|---|---|
| `int x;` | (4), (12), (14) |
| `int y=x+1;` | (4), (13), (29), (30), (33), (34) |
| `if (x>a) then{...}else{...}` | (7), (19), (25), (26) |
| `if (x>=a) then{...}` | (7), (18), (25), (27) |
| `bool x=a==b;` | (4), (13), (15), (25), (28) |
| `do{...}while(a>1);` | (8), (20), (25), (26) |
| `a=(b+10)*b;` | (5), (16), (31), (37) |
| `print(a); / print(a+1);` | (6), (17) |

Toàn bộ chương trình mẫu của đề đều được văn phạm sinh ra. Các chỗ như `x>a` (so sánh `int` với `bool`) hay khai báo lại `x` hợp lệ về cú pháp và sẽ được phát hiện ở giai đoạn phân tích ngữ nghĩa nếu mở rộng sau này.